

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 02 -

FUNDAMENTOS DO DVC (DATA VERSION CONTROL)

SUMÁRIO

O QUE VEM POR AÍ?	3
HANS DS ON	5
SAIBA MAIS	6
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	14
REFERÊNCIAS	15

EMAN

O QUE VEM POR AÍ?

Nesta aula, você verá como o Data Version Control (DVC) expande as funcionalidades do Git para versionamento de dados e modelos em projetos de Machine Learning. Veremos por que o versionamento de datasets é crucial para manter a confiabilidade das métricas de desempenho de modelos, especialmente à medida que os dados evoluem ao longo do tempo e podem alterar distribuições estatísticas e degradar a acurácia das predições. Discutiremos também a importância da rastreabilidade e reprodutibilidade em ambientes regulados (por exemplo, no setor financeiro ou de saúde) – práticas cada vez mais exigidas por normas como a LGPD (Lei Geral de Proteção de Dados) e a GDPR europeia – e como o DVC contribui para auditoria, compliance e validação científica dos experimentos de ML.

Para “quebrar o gelo”, pense no seguinte: e se você pudesse clonar um repositório e ter, instantaneamente, não só o código do seu projeto, mas também todos os datasets e modelos exatamente nas versões usadas para treinar cada modelo? É exatamente isso que o DVC viabiliza, atuando como “Git para dados”. Ao longo deste conteúdo, vamos explorar:

- O DVC versão dados volumosos sem sobrecarregar o Git, usando arquivos de metadados (*.dvc) que apontam para dados mantidos fora do repositório.
- Os efeitos de mudanças de dados nos modelos de ML e por que sem versionamento não é possível reproduzir experimentos nem comparar resultados de forma confiável.
- A integração do DVC em pipelines de MLOps, combinando-o com ferramentas como MLflow (gerenciamento de experimentos) e orquestradores como Kubeflow/Airflow para automatizar retreinamentos e monitorar modelos em produção.
- Casos reais de uso do DVC em empresas de tecnologia, saúde e finanças, mostrando como a ferramenta traz ganhos de produtividade e confiabilidade a projetos de ML.

- Tendências emergentes em versionamento de dados, incluindo a recente aquisição do DVC pela plataforma lakeFS (2025) e as perspectivas de uma futura convergência com outras soluções de MLOps.

Acompanhe-nos nesta jornada sobre o DVC e prepare-se para revolucionar a forma de gerenciar dados e modelos nos seus projetos de Machine Learning. Ao final desta aula, você entenderá como aplicar o DVC para garantir reprodutibilidade, colaborar com sua equipe de forma eficaz e manter o controle das versões dos seus conjuntos de dados e modelos.



HANDS ON

Vamos colocar a teoria em prática! Nesta atividade hands on, demonstraremos um ciclo completo de Machine Learning com ênfase no versionamento de dados e reprodutibilidade usando o DVC. Nas videoaulas práticas correspondentes, você verá passo a passo como configurar um projeto com Git + DVC, adicionar datasets versionados e reproduzir experimentos com consistência. Também exploraremos a integração do DVC a pipelines MLOps, simulando colaboração em equipe com versionamento de artefatos e sincronização de dados em storage remoto.

Requisitos para o hands on: é recomendada a instalação prévia do Python 3.12+ e do DVC (≥ 2.0), além de uma conta em serviço de armazenamento (p. ex., AWS S3, Google Cloud Storage ou Azure Blob). O código completo está disponível em um repositório GitHub para que você possa acompanhar e reproduzir os comandos apresentados. Código completo disponível no GitHub: <https://github.com/Cataldir/Materiais-MLET/tree/main/fase-02-containers-e-ambientes-reprodutíveis/04-dvc-mlflow>.

SAIBA MAIS

À medida que projetos de Machine Learning evoluem, os dados de treinamento e teste estão em constante mudança – seja pela coleta de novos exemplos, correção de valores inconsistentes ou adição de novas features. Essas alterações aparentemente rotineiras podem ter impacto profundo nas distribuições estatísticas dos atributos e, conseqüentemente, nas métricas de desempenho dos modelos.

Por exemplo, um classificador treinado em um certo conjunto de dados pode apresentar queda de acurácia ou F1-score se for avaliado em dados cuja distribuição mudou em relação ao treinamento (um fenômeno conhecido como data drift ou concept drift). Formalmente, se $P_{\text{orig}}(X, y)$ denota a distribuição original dos dados e $P_{\text{novo}}(X, y)$ a nova distribuição após mudanças no dataset, a performance do modelo f_{θ} otimizado em P_{orig} tende a degradar-se quando aplicado em P_{novo} , principalmente se houver deslocamento significativo em $P(X)$ ou $P(y|X)$.

Nem sempre essas mudanças de distribuição são evidentes, o que dificulta a garantia de confiabilidade e generalização dos modelos ao longo do tempo. Segundo Sculley et al. (2015), mesmo sistemas de ML com alto desempenho inicial podem acumular uma “dívida técnica oculta” – custos de manutenção crescentes decorrentes de feedback, loops de dados, dependências invisíveis e drifts de distribuição. Esse acúmulo silencioso de problemas pode corroer a qualidade do modelo e torna praticamente impossível reproduzir resultados antigos se não houver um controle de versão rigoroso dos dados e parâmetros. Por exemplo, mudanças sutis em um feature crucial (como um ajuste em um banco de dados ou a atualização de um modelo auxiliar) podem alterar o comportamento de todo o pipeline, dificultando a rastreabilidade das causas de uma degradação de performance.

Garantir a reprodutibilidade experimental significa que, ao rodar novamente um experimento no futuro, sob as mesmas condições, obtemos resultados idênticos. Em termos formais, se um modelo f_{θ} é treinado para minimizar uma função de perda L em um dataset de treino D_{train} – por exemplo, encontrando

$$\theta^* = \arg\min_{\theta} \frac{1}{|D_{\text{train}}|} \sum_{(x_i, y_i) \in D_{\text{train}}} L(f_{\theta}(x_i), y_i)$$

$D_{\{\text{train}\}}$ $L(f(x_i; \theta), y_i)$ – então para reproduzir o modelo θ^* precisamos do mesmo conjunto $D_{\{\text{train}\}}$ e do mesmo código de treinamento (além de fixar a semente aleatória e demais hiperparâmetros).

Sem versionamento de dados, essa reprodução confiável se torna inviável. Ferramentas como o DVC surgem para suprir exatamente essa necessidade: armazenar e recuperar versões específicas de datasets e modelos, vinculando-as a commits do código e garantindo que possamos reexecutar pipelines passados sob demanda. Essa capacidade é vital não apenas para pesquisa acadêmica – onde a reprodutibilidade científica é mandatória (Kapoor; Narayanan, 2023) – mas também para aplicações industriais que exigem auditoria rigorosa. Por exemplo, em setores como finanças e saúde, reguladores frequentemente exigem que instituições provem como um modelo foi treinado e com quais dados (para atestar fairness, ausência de viés e conformidade regulatória). Sem uma ferramenta de versionamento de dados, equipes de ML enfrentam grandes dificuldades para atender a esses requisitos.

O que é o DVC e como ele funciona?

O DVC (Data Version Control) é uma ferramenta open source, criada em 2017 por Dmitry Petrov, e desenvolvida pela empresa Iterative.ai. Inspirado no Git, o DVC foi projetado para levar as melhores práticas do desenvolvimento de software para os fluxos de trabalho de Machine Learning. Em essência, o DVC estende o Git para gerenciar versões de dados e modelos de ML, resolvendo o “gap” deixado pelos sistemas tradicionais de versionamento, que não lidam bem com arquivos muito grandes ou binários.

Segundo a documentação oficial, o DVC torna os experimentos de ML “compartilháveis e reprodutíveis”, permitindo acompanhar versões de modelos, dados e até pipelines de processamento dentro do repositório. Diferentemente de extensões como o Git LFS (Large File Storage), que armazenam arquivos grandes, mas não integram comandos específicos para ML, o DVC foi pensado desde o início para as necessidades de Cientistas de Dados e Engenheiros de ML, incluindo facilidades para

experimentação, comparações de métricas e integração com fluxos de trabalho de treinamento e implantação.

Arquitetura conceitual do DVC

Podemos visualizar o DVC em três camadas integradas:

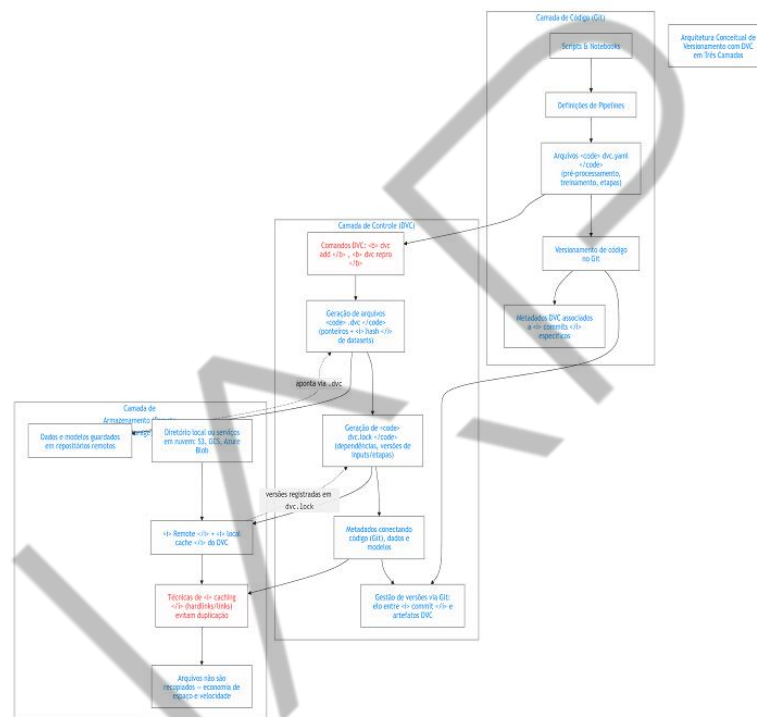


Figura 1 - DVC
Fonte: Elaborado pelo autor (2026)

Camada de Código (Git): onde residem scripts, notebooks e definições de pipelines (por exemplo, arquivos `dvc.yaml` para descrever etapas de pré-processamento e treinamento). O Git continua sendo a base para versionar todo o código e metadados produzidos pelo DVC, garantindo que cada alteração de modelo ou de dados esteja associada a um commit específico.

Camada de Controle (DVC): constitui o cérebro do DVC. Quando você executa `dvc add` ou `dvc repro`, o DVC gera arquivos de metadados (`.dvc` e `dvc.lock`) que representam os dados e modelos versionados, bem como as dependências entre etapas de processamento. Por exemplo, o arquivo `.dvc` aponta para o local de

armazenamento e o hash de um dataset, enquanto o `dvc.lock` guarda quais versões de cada input geraram um certo output. Esses arquivos de controle ficam sob gestão do Git, criando um elo entre o commit de código e as versões dos dados.

Camada de Armazenamento (Remote storage): é onde os dados e modelos em si são guardados. O DVC permite configurar diversos tipos de repositório remoto – desde um simples diretório local compartilhado em rede até serviços em nuvem como o Amazon S3, Google Cloud Storage ou Azure Blob Storage. Os dados versionados são armazenados nesse remote e também em um cache local (na pasta `.dvc/cache`). O DVC utiliza técnicas de caching (por exemplo, hardlinks ou softlinks no sistema de arquivos) para evitar duplicação de dados: um mesmo arquivo de dados não é copiado novamente se já existir no cache, economizando espaço e acelerando as operações.

Uma das grandes sacadas da arquitetura do DVC é que somente os arquivos de metadados são versionados no Git, enquanto os dados ficam fora do repositório. Quando um colaborador clona o repositório e executa `dvc pull`, o DVC lê os arquivos `.dvc` e sabe exatamente quais arquivos baixar do storage remoto para reconstruir o estado do projeto naquele commit. Isso possibilita que equipes trabalhem com datasets massivos (GB ou TB) de forma colaborativa, sem sobrecarregar o repositório Git central e sem perder rastreabilidade.

Reprodutibilidade e experimentação com DVC

Além de versionar dados, o DVC oferece recursos avançados para gerenciar todo o ciclo de experimentação em ML. Um desses recursos é a definição de pipelines reprodutíveis via arquivos `dvc.yaml`. Neles, declaramos etapas (stages) – por exemplo: pré-processamento, treino, validação – especificando comandos a serem executados, suas dependências (scripts, dados de entrada, modelos) e produtos (artifacts) gerados em cada fase.

Ao executar `dvc repro`, o DVC utiliza essas informações para determinar automaticamente quais etapas precisam ser reexecutadas, com base em quais arquivos de entrada foram alterados. Isso implementa o chamado rebuild seletivo: se

you adjust only the preprocessing phase, for example, DVC recognizes that subsequent steps (such as training) must be reexecuted, while previous steps can be reused. In this way, experiments become easy to reproduce and efficient, saving time and computational resources. It is worth noting that DVC allows integrating these pipelines into the development workflow, making it simple to run ML experiments via CLI or even in Continuous Integration/Continuous Delivery (CI/CD) systems.

Another highlight is the support for tracking experiments and metrics. In research and model development projects, it is common to test multiple variations of hyperparameters, network architectures or different data sets. With DVC 2.0 (released in 2021), the concept of “lightweight experiments” (lightweight experiments) emerged, which allows running experiments without “polluting” the Git history with hundreds of temporary commits.

DVC stores the results and differences of these experiments separately, facilitating the comparison of metrics between them (for example, model performance under different learning rates or input features) without compromising the main repository. These comparison and visualization capabilities can be leveraged either by integration with IDEs (such as the DVC extension for VS Code) or by generating charts and tables of results (such as the visualization of metrics plots during training, through the DVCLive module).

Next, we illustrate how DVC differs from traditional approaches such as manual training or non-versioned scripts in terms of reproducibility and tracking.

Aspecto	Fluxo Tradicional (Sem DVC)	Fluxo com DVC
Versionamento de dados	Feito manualmente (sujeito a erro e esquecimento).	Automatizado via <code>dvc add</code> (metadados <code>.dvc</code>).
Reprodutibilidade	Difícil de garantir; dependente de anotações manuais sobre quais dados e parâmetros foram usados.	Garantida por design; cada experimento é atrelado aos dados e parâmetros exatos via <code>commit</code> do Git.
Comparação de métricas	Feita manualmente, propensa a confusão de versões.	Facilitada pelo DVC (<code>dvc metrics diff</code>), que exibe diferenças de métricas entre versões de experimentos.
Colaboração em equipe	Compartilhamento de dados por meios ad-hoc (ex.: planilhas, e-mail), risco de desatualização.	Colaborativo e centralizado; dados compartilhados via storage unificado, com histórico e <i>logs</i> claros (ex.: <code>dvc push/pull</code>).
Integração em MLOps	Dificuldade para vincular código (Git), dados e pipelines de forma coesa.	Perfeitamente integrável com CI/CD e orquestradores (Airflow, Kubeflow), unificando código, dados e execução em pipelines rastreáveis.

Figura 2 - Comparativo entre abordagem tradicional e abordagem com DVC para experimentos de ML

Fonte: Elaborado pelo autor (2026)

O DVC atua como um pilar de sustentação da engenharia de Machine Learning moderna, servindo como uma camada de controle que mitiga a dívida técnica de dados e promove práticas de MLOps (Machine Learning Operations) robustas (Hewage; Meedeniya, 2022). Ao adotar o DVC, equipes de ML podem focar em experimentação e inovação, confiando que cada resultado poderá ser auditado e reproduzido no futuro.

MERCADO, CASES E TENDÊNCIAS

Cases de sucesso e adoção no mercado: desde seu lançamento, o DVC tem conquistado ampla adoção na comunidade de ciência de dados e aprendizagem de máquina. O projeto open-source acumula mais de 15 mil estrelas no GitHub, indicando sua popularidade e confiança na indústria (Treeverse, 2026). Empresas de tecnologia com grandes volumes de dados relatam o uso do DVC para gerenciar datasets de treino e garantir que equipes distribuídas trabalhem sempre com os mesmos dados, evitando o “caos de versões” que antes atrasava projetos críticos de IA (Iterative, 2025).

No setor financeiro, fintechs e bancos passaram a adotar o DVC para versionar bases de dados de históricos de crédito e detecção de fraudes, assegurando que modelos preditivos possam ser auditados e recriados sob demanda para fins de compliance e investigação interna. Na área médica e de saúde, a rastreabilidade completa provida pelo DVC tem sido utilizada para obter aprovação regulatória de modelos de diagnóstico: instituições de pesquisa conseguem demonstrar todos os passos – do conjunto de dados original aos resultados finais – provando que suas previsões são reproduzíveis e confiáveis (Kapoor; Narayanan, 2023).

Publicações e recursos recentes: a literatura técnica tem acompanhado de perto o avanço do DVC e de ferramentas correlatas. Uma pesquisa sobre o suporte de ferramentas em MLOps (Hewage; Meedeniya, 2022) destacou o DVC como uma das soluções proeminentes para gerenciamento de artefatos de ML. Já um estudo empírico apresentado no IEEE SANER 2021 (Barrak; Eghan; Adams, 2021) investigou repositórios públicos usando DVC e revelou que projetos com pipelines definidos em DVC apresentaram maior facilidade de manutenção e refatoração de código de ML, evidenciando o valor dessa coevolução controlada entre código e dados.

Além disso, a comunidade MLOps vem produzindo diversos guias e comparativos: por exemplo, whitepapers de engenheiros do Google e Amazon abordam como ferramentas de versionamento de dados (DVC, Pachyderm, lakeFS etc.) contribuem para a escalabilidade de soluções de IA corporativas (Google, 2024). Podcasts e vídeos técnicos também merecem menção – destacamos o episódio

“MLOps Podcast: Reproducibility in ML” (2025), com engenheiros da Netflix e Amazon discutindo a adoção de DVC em produções de larga escala.

A palestra “Data Versioning for Reproducible AI”, apresentada na conferência NeurIPS 2024, enfatiza as melhores práticas para controle de versão de dados em pesquisa acadêmica. Esses materiais ampliam a visão sobre como diferentes organizações estão enfrentando os desafios de versionamento e compartilhando lições aprendidas.

Tendências e próximas evoluções: no horizonte próximo, o versionamento de dados consolida-se como prática padrão em MLOps. A recente aquisição do DVC pela empresa Treeverse (criadora do lakeFS), em novembro de 2025, sinaliza um movimento de unificação de ferramentas de governança de dados. Essa fusão promete aproximar o melhor dos dois mundos: workflows baseados em Git (DVC) e arquiteturas nativas de data lakes (lakeFS), criando ecossistemas integrados para dados e modelos.

Outra tendência forte é a integração do DVC com plataformas de orquestração e tracking: cada vez mais times combinam o DVC com o MLflow (para rastrear experimentos e conferir modelos em um registry), com o Apache Airflow (para automação de pipelines de dados) e com o Kubeflow (para implementar pipelines de ML em Kubernetes). Ferramentas de AutoML e frameworks de treinamento distribuído também têm incorporado funcionalidades de versionamento de dados, muitas vezes construídas em cima de princípios similares aos do DVC (Gartner, 2024).

Por fim, observa-se um crescente enfoque em Data Governance e DataOps no contexto de IA: empresas buscam não apenas desenvolver modelos acurados, mas também garantir transparência, auditabilidade e confiabilidade de cada etapa do ciclo de vida dos modelos. Nesse cenário, o DVC e suas ferramentas correlatas se firmam como componentes essenciais para uma infraestrutura de IA robusta e preparada para o futuro.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você entendeu por que versionar dados é tão importante quanto versionar código em projetos de Machine Learning, sendo fundamental para reprodutibilidade, auditoria e melhoria contínua dos modelos. Vimos que o DVC estende o Git para lidar com datasets e modelos de ML, usando metafiles (.dvc, dvc.lock) que apontam para os dados armazenados externamente, evitando sobrecarga no repositório e facilitando a colaboração em equipe.

Aprendemos a configurar o DVC em um repositório Git, adicionando datasets com `dvc add` e sincronizando arquivos para um storage remoto via `dvc push/pull`, o que habilita o compartilhamento de grandes datasets de forma controlada. Compreendemos ainda como o DVC assegura reprodutibilidade de experimentos por meio de `dvc checkout` e `dvc repro`, garantindo que um modelo possa ser retreinado exatamente com os mesmos dados e sequências de passos originais, permitindo comparações justas de métricas entre versões.

Aprofundamos os fundamentos teóricos discutindo a influência da evolução dos dados no desempenho dos modelos, apresentando o conceito de dívida técnica em ML e detalhando a arquitetura e recursos do DVC (incluindo pipelines e experimentos).

Vimos casos reais e tendências de mercado relacionadas ao DVC e MLOps com exemplos de empresas de diversos setores usando DVC, bem como o panorama em evolução, como a união do DVC com o lakeFS (2025) e a integração com tecnologias como MLflow e Kubeflow para criar pipelines de ML mais robustos.

Em suma, esta aula proporcionou uma visão 360° sobre o Data Version Control: da motivação teórica (por que precisamos versionar dados) às práticas de implementação (como usar DVC na sua máquina e em projetos colaborativos), passando pelo contexto de mercado e as tendências que moldam o futuro da engenharia de Machine Learning.

REFERÊNCIAS

BARRAK, A.; EGHAN, E. E.; ADAMS, B. **On the co-evolution of ML pipelines and source code – Empirical study of DVC projects**. In: Proc. of 28th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 422–433, 2021. DOI: 10.1109/SANER50967.2021.00046.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. MIT Press, 2016. Disponível em: <https://www.deeplearningbook.org>. Acesso em: 18 mar. 2026.

GOOGLE CLOUD. **MLOps: Continuous delivery and automation pipelines in machine learning**. Cloud Architecture Center, 2024. Disponível em: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>. Acesso em: 18 mar. 2026.

HEWAGE, N.; MEEDENIYA, D. **Machine Learning Operations: A Survey on MLOps Tool Support**. arXiv preprint arXiv:2202.10169, 2022. Disponível em: <https://arxiv.org/abs/2202.10169>. Acesso em: 18 mar. 2026.

ITERATIVE AI. **DVC Documentation – Get Started**. Iterative.ai, 2025. Disponível em: <https://dvc.org/doc>. Acesso em: 18 mar. 2026.

KAPOOR, S.; NARAYANAN, A. **Leakage and the reproducibility crisis in ML-based science**. Patterns, v.4, n.9, p.100804, 2023. DOI: 10.1016/j.patter.2023.100804.

ORR, E. **Data versioning as your “Get out of jail” card – DVC vs. Git-LFS vs. Dolt vs. lakeFS**. lakeFS Blog, 05 jan. 2026. Disponível em: <https://lakefs.io/blog/dvc-vs-git-vs-dolt-vs-lakefs/>. Acesso em: 18 mar. 2026.

SCULLEY, D. et al. **Hidden Technical Debt in Machine Learning Systems**. In: Advances in Neural Information Processing Systems 28 (NIPS 2015). Disponível em: <https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf>. Acesso em: 18 mar. 2026.

SELWAN, R. **DVS and Data Pipeline Using Airflow, Kubeflow and MLflow**. LinkedIn Pulse, 22 abr. 2025. Disponível em: <https://www.linkedin.com/pulse/dvs-data-pipeline-using-airflow-kubeflow-mlflow-selvan-rajani/>. Acesso em: 18 mar. 2026.

TREEVERSE (lakeFS.io). **lakeFS Acquires DVC, Uniting Data Version Control Pioneers to Accelerate AI-ready Data**. Comunicado à Imprensa, 15 nov. 2025. Disponível em: <https://lakefs.io/blog/lakefs-acquires-dvc/>. Acesso em: 18 mar. 2026.

PALAVRAS-CHAVE

Versionamento de Dados. Reprodutibilidade em Machine Learning. MLOps.

EMENDAS

The background is a dark, abstract network visualization. It features a complex web of interconnected nodes and lines. The nodes are represented by small circles in various colors, including teal, orange, and grey. The lines connecting them are thin and multi-colored, creating a sense of depth and connectivity. The overall effect is a futuristic, digital landscape.

POSTECH